1. Use only participants already verified in the frozen baseline and `patterns.csv`.
2. Do not invent interfaces, abstract classes or methods just to make the diagram resemble a textbook pattern.
3. Show only the important participants needed to understand the collaboration.
4. Label relationships with the actual responsibility or interaction where useful.
5. For GRASP Controller diagrams, show the system-event coordinator and its important collaborators. Do not invent GoF-style Controller interfaces.
6. For the Information Expert diagram, make it clear that `InboxItem` owns the parsed data and performs the record-linking responsibility while collaborating with `TripItem`, `Expense`, `Trip` and the requesting controller.
7. For Outbound Mail Delivery Logging, clearly distinguish:
   DaPlanStan `MailLogger`
   Action Mailer registration
   Mail framework notification infrastructure
   mail-producing collaborators.
8. For Trip-Scoped Turbo Refresh Notifications, clearly distinguish:
   application publication sources
   trip-stream subscription declarations
   Turbo Streams
   Action Cable
   browser subscriber/refresh behavior.
9. Do not represent framework participants as application-written classes.
10. Do not add rejected candidates such as geocoding Strategy, Template Method, Adapter or timeline-map Observer.

For each diagram give:

1. A short diagram title.
2. Mermaid source.
3. A short explanation of what the diagram proves about the identified pattern.
4. The corresponding frozen pattern ID P01–P09.

Keep the diagrams readable rather than showing every method in every class.
At the end provide a short checklist confirming:

1. 9 diagrams produced.
2. All frozen instances represented once.
3. No rejected candidates included.
4. No invented participants.
5. Application and framework roles distinguished.

Do not perform evolution analysis yet.

DaPlanStan — Editable UML Diagrams

Frozen baseline · 9 confirmed instances · Mermaid UML source

The nine diagrams below use the frozen source-supported inventory. They show the actual application participants and their collaborations without inventing pattern interfaces, abstract classes or additional pattern instances.

Notation: `<<application>>` identifies DaPlanStan code; `<<framework>>` identifies external framework participants; dashed arrows indicate calls, dependencies or notifications; solid arrows show established domain relationships. View templates are represented as views rather than invented JavaScript classes.

P01 — Parsed Email to Trip Record Linking

Information Expert (GRASP)